Experiment Title: Implementation of First Come, First Served CPU Scheduling Algorithm in C++

Topic: Non-Preemptive FCFS CPU Scheduling Simulation with Arrival Time in C++

1. Problem

In operating systems, process scheduling decides the order of process execution on the CPU. The objective of this lab is to implement the **First Come, First Served (FCFS)** CPU scheduling algorithm in C++.

The program must:

- Take the number of processes (n), arrival time (AT), and execution/burst time (BT) for each process.
- Validate input data (positive count, non-negative arrival time, and positive execution time).
- Schedule processes strictly in order of their arrival time using stable sorting.
- Compute Start Time (ST), Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT).
- Calculate and display Average Waiting Time and Average Turnaround Time.
- Display the execution schedule table and text-based Gantt chart timeline.

1.1 Environment & Tools

Item	Details	Item	Details
Operating System	Linux (Ubuntu 24.04 LTS)	**Host Machine**	`firefly`
User Account	`ratul`	**Language & Standard**	C++ (GNU C++17)
Compiler	GCC / G++ (`g++ 13.x`)	**Execution Shell**	GNU Bash 5.2

2. Theoretical Solution

First Come, First Served (FCFS) is the simplest non-preemptive CPU scheduling algorithm. The process that arrives first in the ready queue is allocated the CPU first.

2.1 Key Principles

- **FIFO Queue Discipline:** Processes are serviced strictly in arrival sequence.
- **Non-Preemptive Execution:** Once a process gets the CPU, it runs to completion without interruption.
- **Tie-Breaking:** When arrival times match, input order is maintained via stable sorting.
- **CPU Idle Handling:** If the CPU finishes before the next process arrives, current time jumps forward: $ST_i = \max(currentTime, AT_i)$.

2.2 Formulas

For each process i, scheduling metrics are computed as follows:

- **Start Time (ST):** $ST_i = \max(currentTime, AT_i)$
- **Completion Time (CT):** $CT_i = ST_i + BT_i$
- **Turnaround Time (TAT):** $TAT_i = CT_i - AT_i$
- **Waiting Time (WT):** $WT_i = ST_i - AT_i = TAT_i - BT_i$
- **Average Turnaround Time (Avg_TAT):** $\frac{1}{n}\sum_{i=1}^{n} TAT_i$
- **Average Waiting Time (Avg_WT):** $\frac{1}{n}\sum_{i=1}^{n} WT_i$

2.3 Gantt Chart Representation

Figure 1 illustrates the non-preemptive FCFS timeline for 4 processes (P_1: $AT = 0, BT = 4$; P_2: $AT = 1, BT = 3$; P_3: $AT = 2, BT = 1$; P_4: $AT = 3, BT = 2$):

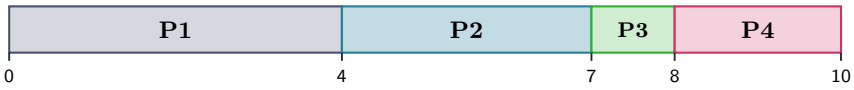


Figure 1: Gantt Chart of Non-Preemptive FCFS Execution.

2.4 Algorithmic Approach

1. Read process count n and validate that $n > 0$.
2. Read arrival (AT) and execution (BT) times; validate $AT \geq 0$ and $BT > 0$.
3. Sort processes in ascending order of arrival time using `std::stable_sort`.
4. For each process i: update $currentTime = \max(currentTime, AT_i)$, set $start = currentTime$, compute $waiting = start - AT_i$, advance $currentTime = currentTime + BT_i$, and record $completion = currentTime$ and $turnaround = completion - AT_i$.
5. Sort back by process ID, print results table, display averages, and output Gantt chart.

3. Expected Input/Output (I/O) Set

The input consists of integer counts, arrival times, and execution times. The output displays the detailed schedule, averages, and the Gantt chart sequence.

3.1 Input Specification

- Total processes: $n = 4$
- Input pairs: Process ID, Arrival Time (AT), Execution Time (BT).

3.2 Tabular Input and Expected Output

Process	Arrival (AT)	Execution (BT)	Start (ST)	Completion (CT)	Turnaround (TAT)	Waiting (WT)
P1	0	4	0	4	4	0
P2	1	3	4	7	6	3
P3	2	1	7	8	6	5
P4	3	2	8	10	7	5

3.3 Expected Summary Metrics

- **Total Waiting Time:** $0 + 3 + 5 + 5 = 13$
- **Total Turnaround Time:** $4 + 6 + 6 + 7 = 23$
- **Average Waiting Time:** $13/4 = 3.25$
- **Average Turnaround Time:** $23/4 = 5.75$
- **Gantt Chart Sequence:** `| P1 | P2 | P3 | P4 |` with time points $0, 4, 7, 8, 10$.

4. Source Code

The complete C++ implementation of the First Come, First Served CPU scheduling algorithm is given below:

```cpp
#include <bits/stdc++.h>
using namespace std;

struct Process {
    int id;
    int arrival;
    int execution;
    int start;
    int completion;
    int turnaround;
    int waiting;
};

int main() {
    int n;

    cout << "Enter the number of processes: ";
    cin >> n;

    if (cin.fail() || n <= 0) {
        cout << "Invalid number of processes!\n";
        return 1;
    }

    vector<Process> processes(n);

    // Input process details
    for (int i = 0; i < n; i++) {
        processes[i].id = i + 1;

        cout << "Enter arrival and execution time for process "
             << processes[i].id << ": ";

        cin >> processes[i].arrival >> processes[i].execution;

        if (cin.fail() ||
            processes[i].arrival < 0 ||
            processes[i].execution <= 0) {
            cout << "Invalid input! Arrival time must be "
                 << "non-negative and execution time must "
                 << "be positive.\n";
            return 1;
        }
    }

    // Sort by arrival time; preserve input order for ties
    stable_sort(processes.begin(), processes.end(),
        [](const Process& a, const Process& b) {
            return a.arrival < b.arrival;
        });

```

```
    int currentTime = 0;
    double totalWaiting = 0;
    double totalTurnaround = 0;

    // Calculate scheduling times
    for (Process& p : processes) {
        // Handle CPU idle time
        currentTime = max(currentTime, p.arrival);

        p.start = currentTime;
        p.waiting = p.start - p.arrival;

        currentTime += p.execution;

        p.completion = currentTime;
        p.turnaround = p.completion - p.arrival;

        totalWaiting += p.waiting;
        totalTurnaround += p.turnaround;
    }

    // Display results in original process order
    sort(processes.begin(), processes.end(),
        [](const Process& a, const Process& b) {
            return a.id < b.id;
        });

    cout << "\nProcess\tArrival\tExecution\tStart\tCompletion"
         << "\tTurnaround\tWaiting\n";

    for (const Process& p : processes) {
        cout << "P" << p.id << "\t"
             << p.arrival << "\t"
             << p.execution << "\t\t"
             << p.start << "\t"
             << p.completion << "\t"
             << p.turnaround << "\t\t"
             << p.waiting << '\n';
    }

    // Display average times
    cout << fixed << setprecision(2);

    cout << "\nAverage Waiting Time: "
         << totalWaiting / n << '\n';

    cout << "Average Turnaround Time: "
         << totalTurnaround / n << '\n';

    // Display Gantt chart in FCFS execution order
    stable_sort(processes.begin(), processes.end(),
        [](const Process& a, const Process& b) {
            return a.arrival < b.arrival;
        });

    cout << "\nGantt Chart:\n";

    for (const Process& p : processes) {
        if (p.start > 0 && p.start > p.arrival) {
            // The CPU may have been idle before this process.
            // Idle intervals are displayed below using timestamps.
        }

        cout << "| ";
        if (p.start > 0) {
            cout << "P" << p.id << " ";
        } else {
            cout << "P" << p.id << " ";
        }
    }
    cout << "|\n";

    // Print process start and completion timestamps
    if (!processes.empty()) {
        cout << processes[0].start;

        for (const Process& p : processes) {
            cout << "\t" << p.completion;
        }

        cout << '\n';
    }

    return 0;
}
```

5. Execution of Code

The C++ program was compiled using G++ and executed in the Linux terminal environment.

5.1 Compilation and Terminal Output

```
ratul@firefly: ~/OS-Lab — bash

ratul@firefly:~/OS-Lab$ g++ fcfs.cpp -o fcfs
ratul@firefly:~/OS-Lab$ ./fcfs
Enter the number of processes: 4
Enter arrival and execution time for process 1: 0 4
Enter arrival and execution time for process 2: 1 3
Enter arrival and execution time for process 3: 2 1
Enter arrival and execution time for process 4: 3 2
Process  Arrival  Execution  Start  Completion  Turnaround  Waiting
P1       0        4          0      4           4           0
P2       1        3          4      7           6           3
P3       2        1          7      8           6           5
P4       3        2          8      10          7           5
Average Waiting Time: 3.25
Average Turnaround Time: 5.75
Gantt Chart:
| P1 | P2 | P3 | P4 |
0    4    7    8    10
ratul@firefly:~/OS-Lab$ _
```

5.2 Verification of Correctness

The execution output confirms accurate First Come, First Served scheduling:

- **Process P_1:** Arrives at $T = 0$. Starts immediately at $T = 0$, runs for 4 units, and finishes at $T = 4$. $WT = 0, TAT = 4$.
- **Process P_2:** Arrives at $T = 1$. Waits until P_1 completes at $T = 4$. Runs for 3 units until $T = 7$. $WT = 4 - 1 = 3, TAT = 7 - 1 = 6$.
- **Process P_3:** Arrives at $T = 2$. Starts at $T = 7$, runs for 1 unit until $T = 8$. $WT = 7 - 2 = 5, TAT = 8 - 2 = 6$.
- **Process P_4:** Arrives at $T = 3$. Starts at $T = 8$, runs for 2 units until $T = 10$. $WT = 8 - 3 = 5, TAT = 10 - 3 = 7$.
- The generated textual Gantt chart sequence `| P1 | P2 | P3 | P4 |` with boundary timestamps $0, 4, 7, 8, 10$ matches theoretical calculations identically.

6. Conclusion

Key takeaways from this lab:

1. Implemented the First Come, First Served (FCFS) CPU scheduling algorithm in C++.
2. Handled arbitrary process arrival times, CPU idle gaps, and preserved arrival order using stable sorting.
3. Computed process metrics (ST, CT, TAT, WT) and printed the scheduling summary and Gantt chart timeline.